Lab 1: SLAM for Karel in 1D

Team 8: Piyush Tiwari, Diego Méndez Carter February 12, 2026

This report presents the implementation of a Simultaneous Localization and Mapping (SLAM) system for a robot operating in a 1D environment. Using a Kalman Filter (KF), we estimate the robot's trajectory and the location of landmarks (beepers). We further explore the implementation of Sequential Map Joining to optimize computational costs when dealing with large-scale maps.

The objective of this assignment is to solve the SLAM problem for "Karel", a robot moving in a one dimensional world. The system must handle:

- **Motion:** Noisy odometry readings.
- **Sensing:** Range measurements to features (beepers) with limited visibility.
- **Data Association:** Distinguishing between new and previously mapped features.

The simulation parameters include a motion noise factor of 10% (σ_x) and a sensor noise factor of 1% (σ_z).

1 KF SLAM Implementation

1.1 Function `add_new_features`

When the sensor sees a feature for the first time, we need to figure out where it is and how sure we are about it but **where is this feature?** If we think we are at the position $\hat{x}_R$ and we measure a distance z to the feature, then the feature is represented as: $\hat{x}_f = \hat{x}_R + z$.

Now, speaking about **uncertainity**. The error comes from two places: we don't know exactly where we are (P_{RR}), and our measurement has noise (R). Since both errors are independent, we can add them up: $P_{ff} = P_{RR} + R$.

While we are advancing, we will be able to see **multiple new features at once.** They all use the same position from which we are to get initialized, so they all share our position error. That makes them correlated with each other through P_{RR}. But each one has its own measurement noise, which is independent. So the covariance between two new features i and j is: $\text{Cov}(f_i, f_j) = P_{RR} + R_i$ if $i = j$, or $\text{Cov}(f_i, f_j) = P_{RR}$ if $i \neq j$. Which we can write compactly as $P_f = P_{RR} \cdot \mathbf{1}_{n\times n} + R_n$.

How do the new features relate to the old map? A new feature inherits whatever relationship the robot had with the rest of the map. If the robot was correlated with some old feature, the new feature will be too, because it was born from the robot's position. So we just copy the robot's column of $\hat{P}$ for each new feature: $P_{of} = \hat{P}_{:,1} \cdot \mathbf{1}_n^T$.

Putting it all together, we grow the covariance matrix by adding blocks:

$$\hat{P} \leftarrow \begin{bmatrix} \hat{P}_{old} & P_{of} \\ P_{of}^T & P_f \end{bmatrix} \tag{1}$$

1.2 Function `update_map`

When the sensor watch again features that are already in the map, we can use those measurements to correct the entire state. This is the standard Kalman filter update.

What we are actually measuring is the distance from the robot to a feature already in the map: $\hat{z}_i = \hat{x}_{f_i} - \hat{x}_R$. **but, did we?** z_i. The difference between what we expected and what we got is the innovation: $y = z - \hat{z}$. If y is large, our estimates were off and need correction.

How do we express this as a matrix equation? For m reobserved features, we need to linearize the model. The measurement to a feature i is $z_i = x_{f_i} - x_R$. Taking partial derivatives with respect to each element of the state vector: $\frac{\partial z_i}{\partial x_R} = -1$, $\frac{\partial z_i}{\partial x_{f_i}} = +1$ for $j \neq i$. These partial derivatives form the Jacobian matrix $H \in \mathbb{R}^{m \times n_{states}}$, where each row has -1 in the robot column and $+1$ in the column of the corresponding observed feature, with zeros everywhere else. This encodes "measure = feature minus robot" for each observation. The full observation model is then: $\mathbf{z} = H\hat{\mathbf{x}} + \mathbf{v}$.

How much should we trust the correction? That depends on the Kalman gain K, which balances how uncertain our state is versus how noisy the measurements are: $S = H\hat{P}H^T + R_f$ and $K = \hat{P}H^T S^{-1}$. If our state is very uncertain and the measurements are precise, K will be large (trust the measurements). If the measurements are noisy and we are already confident, K will be small (mostly ignore them).

Correcting the state: We multiply the Kalman gain K by the innovation y (the "prediction error"). If our prediction was off, Ky pushes the entire state in the right direction. If our prediction was accurate, $y \approx 0$ and almost nothing changes: $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + Ky$.

Correcting the covariance uses the Joseph form, which is numerically more stable than the standard $(I - KH)\hat{P}$:

$$\hat{P} \leftarrow (I - KH)\hat{P}(I - KH)^T + KR_fK^T \tag{2}$$

In the end, when we reobserve a feature, the correction does not just improve that feature's estimate, it improves **everything** in the map that was correlated with it, including the robot position and other features. On Fig. 1, the diagonal non-zero elements (colors different from dark blue) represent the strong correlations between the robot and the mapped features. These correlations allow the KF to correct the position of unobserved features when the robot relocalizes.

2 Sequential Map Joining

Building a single large map has a computational problem: the covariance matrix $\hat{P}$ grows with every new feature, and the Kalman filter update has cost $O(n^2)$. Eventually it becomes too slow. To improve it, we can build **small local maps** and then **join them** into a global map.

Data from our experiment with $N = 4000$ steps (Table 1) confirms this: the single map approach took over 40 minutes to complete, with the final steps taking more than 2.5 seconds each.

In the main loop algorithm creates N local maps sequentially. The first local map becomes the global map. For each subsequent local map, the algorithm:

1. Saves the current robot uncertainty $\sigma_{before} = \sqrt{\hat{P}(1,1)}$ and estimation error $e_{before} = x_R^{true} - \hat{x}_R$ before joining.

2. Joins the global and local maps using the `join_maps` function.

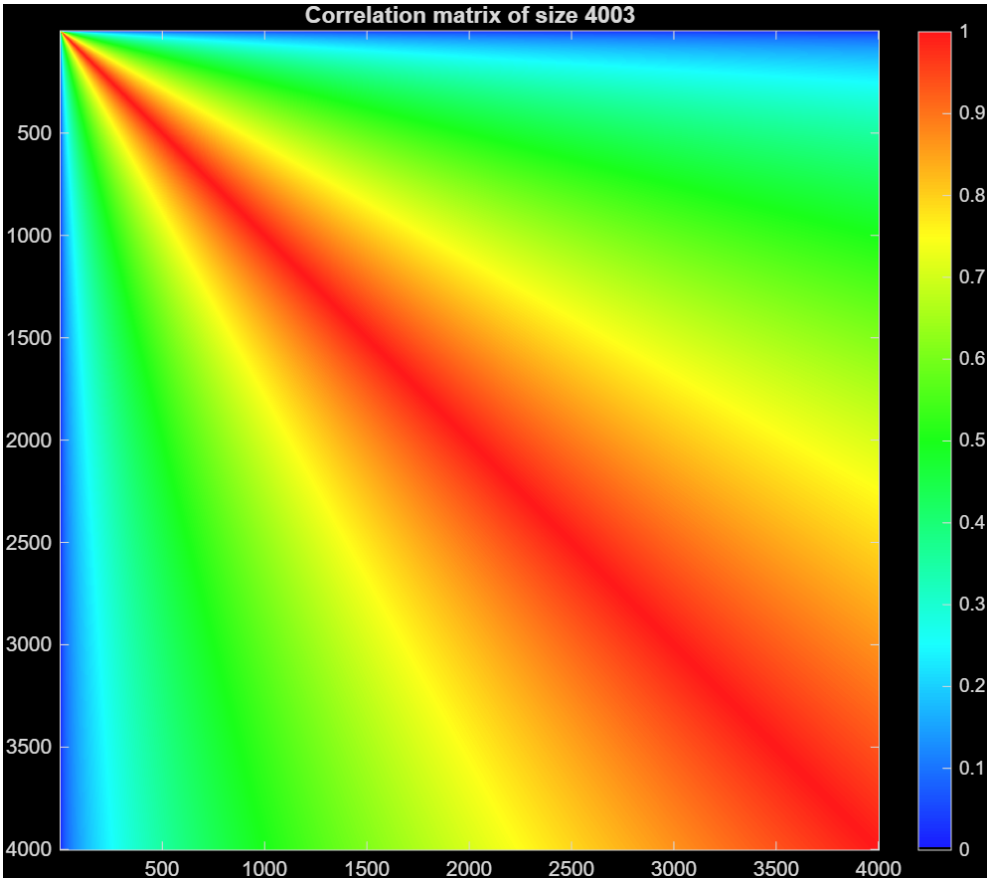


Figure 1: Correlation matrix of the system state.

3. Concatenates the metadata: feature IDs, ground truth positions, and trajectory statistics, applying the appropriate offsets to express everything in the global reference frame.

At the `join_maps` function, each local map has its own reference frame, starting with the robot at position zero. To join map B into map A, we need to express everything in the same frame.

How do we combine the state vectors? The robot in map B starts where the robot in map A ended. So the joined state is:

$$\hat{\mathbf{x}}_{joined} = \begin{bmatrix} \hat{x}_R^A + \hat{x}_R^B \\ \hat{\mathbf{x}}_{features}^A \\ \hat{x}_R^A + \hat{\mathbf{x}}_{features}^B \end{bmatrix} \tag{3}$$

The robot position is the sum of both robot positions. Features from map A stay as they are. Features from map B get shifted by $\hat{x}_R^A$ to move them into map A's reference frame. **How do we combine the covariances?** Since both maps were built independently, their states are uncorrelated. The joint covariance before transformation is block-diagonal:

$$P_{AB} = \begin{bmatrix} P_A & 0 \\ 0 & P_B \end{bmatrix} \tag{4}$$

The joined state is a linear function of the original states, so we propagate uncertainty through the Jacobian J:

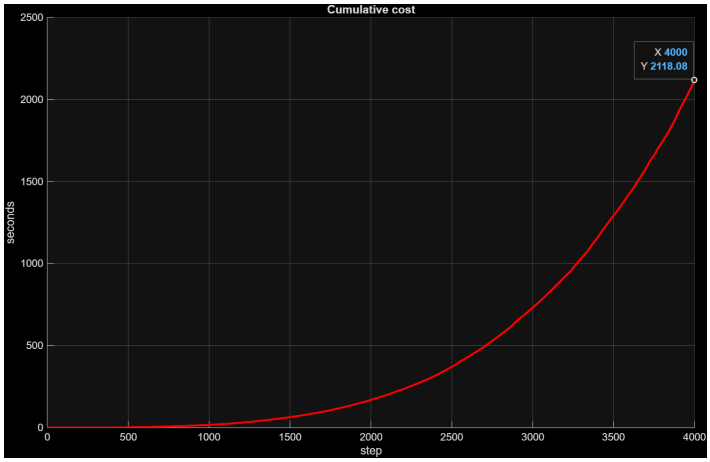


Figure 2: Total map cost (1 map, 4000 steps).

Table 1: Performance Metrics

Metric	**Value**
Total Time	2118.08 s
Slowest Step	2.56 s
Matrix Size	4003

$$P_{joined} = J\, P_{AB}\, J^T \tag{5}$$

The jacobian is constructed by each row of J contains the partial derivatives of one element of $\hat{\mathbf{x}}_{joined}$ with respect to the original states $[\hat{\mathbf{x}}^A, \hat{\mathbf{x}}^B]$:

- **Joined robot position** $= \hat{x}_R^A + \hat{x}_R^B$: derivative is 1 with respect to both $\hat{x}_R^A$ and $\hat{x}_R^B$.
- **Features from map A**: they only depend on themselves, so derivative is 1 in their own column and 0 everywhere else.
- **Features from map B** $= \hat{x}_R^A + \hat{x}_{f_i}^B$: derivative is 1 with respect to $\hat{x}_R^A$ and 1 with respect to $\hat{x}_{f_i}^B$.

This means features from map B inherit the uncertainty of $\hat{x}_R^A$ (the connection point between both maps), just as new features inherit the robot's uncertainty when they are initialized.

The entire operation is matrix-based with no `for` loops, and the use of local maps keeps the covariance matrices small during SLAM, making the algorithm scalable to large environments.

3 Computational Comparison

As stated in the map joining section, the computational cost of SLAM can be significantly reduced by splitting the trajectory into smaller local maps, following a divide and conquer approach. Table 2 shows the impact of this strategy. All configurations cover the same total distance ($N = 4000$ steps), but splitting the trajectory into smaller local maps dramatically reduces computation time while the final matrix size remains nearly the same.

The final covariance matrix size is nearly identical across all configurations (∼4003–4041), confirming that sequential map joining produces the same result as the full approach. The small increase with more maps is due to duplicate features at map boundaries, which could be resolved through data association during the join step.

As shown in Figure 3, the time of execution decreases as long as the number of maps increases. This is because the most expensive operation in the KF update is matrix inversion, whose cost grows cubically

Table 2: Computational cost comparison for different numbers of local maps, all covering 4000 total steps.

n_maps	Steps/map	Total time (s)	Slowest step (s)	Final $\hat{P}$ size
1	4000	2118.08	2.5674	4003
2	2000	303.12	0.3551	4005
4	1000	53.45	0.0530	4009
8	500	11.14	0.0247	4017
20	200	2.61	0.0142	4041

with state size. When we split into n local maps, each map only needs to invert matrices that are n times smaller, making each inversion n^3 times cheaper. Since we build n maps, the total cost is reduced by a factor of n^2.

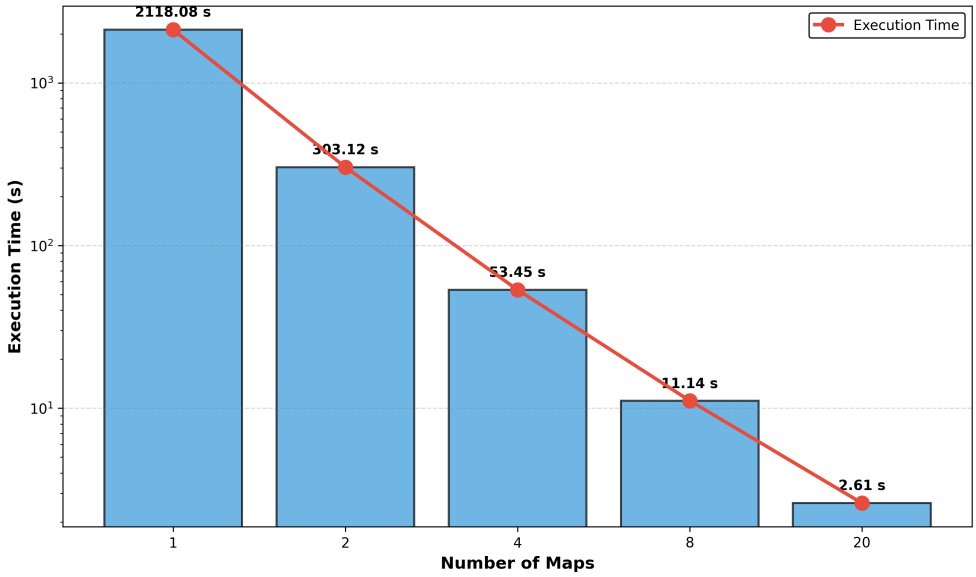


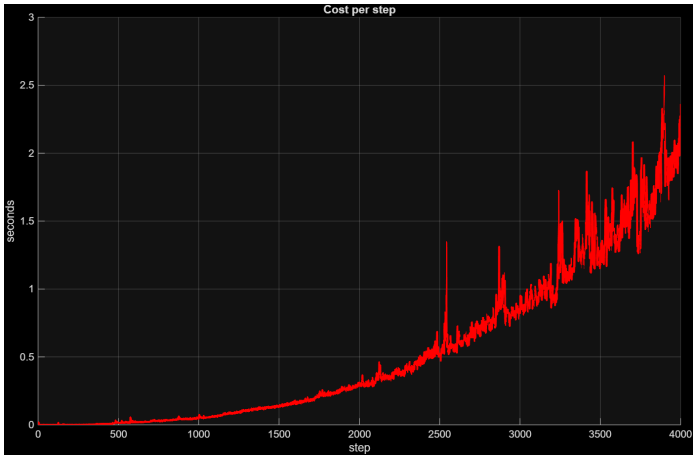
Figure 3: Number of maps vs Time of execution

Figure 4 illustrates the difference in cost per step. In the 1 map experiment, the cost grows continuously as the state vector increases. In the sequential approach (right), the cost resets every time a new local map is started, producing a sawtooth pattern where each peak is much smaller than the full map's final cost.

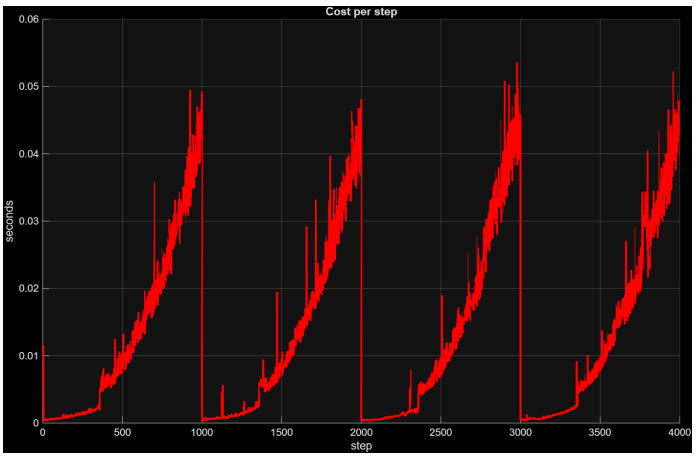
4 Duplicate Feature Fusion

After joining two local maps, the same environmental landmark may appear twice in the state vector: once from the global map ($\mathbf{x}_{F_i}$) and once from the local map ($\mathbf{x}_{F_j}$). Each estimate was computed independently using different sensor observations, so both contain valuable information. Simply discarding one would waste half the available data.

The solution is to apply the same KF update mechanism already used throughout the SLAM process, but now treating the constraint "both features must be at the same location" as if it were a perfect sensor measurement.



(a) Full SLAM (1×4000)



(b) Sequential map joining (4×1000)

Figure 4: Cost per step comparison. In (a), the cost grows continuously as the state vector increases. In (b), the cost resets at each new local map

4.1 Mathematical Formulation

When two features represent the same landmark, we know with certainty that their positions should coincide. This geometric constraint can be expressed as:

$$h_{ij}(\mathbf{x}) = \mathbf{x}_{F_i} - \mathbf{x}_{F_j} = \mathbf{0} \tag{6}$$

We process this constraint exactly like a sensor observation, but with zero measurement noise ($R = 0$). The observation Jacobian encodes "feature i minus feature j":

$$H = \begin{bmatrix} 0 & \cdots & \underbrace{+1}_{\text{pos } F_i} & \cdots & \underbrace{-1}_{\text{pos } F_j} & \cdots & 0 \end{bmatrix} \tag{7}$$

The innovation measures how much the two estimates currently disagree:

$$y = \mathbf{0} - (\hat{\mathbf{x}}_{F_i} - \hat{\mathbf{x}}_{F_j}) = \hat{\mathbf{x}}_{F_j} - \hat{\mathbf{x}}_{F_i} \tag{8}$$

The standard KF update equations then combine both estimates:

$$S = H\hat{P}H^T \tag{9}$$

$$K = \hat{P}H^T S^{-1} \tag{10}$$

$$\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + Ky \tag{11}$$

$$\hat{P} \leftarrow (I - KH)\hat{P}(I - KH)^T \tag{12}$$

These are the same equations used to process sensor measurements during local map building. The only difference is using the Joseph form in Eq. (12) for numerical stability when $R = 0$, and that H now encodes a geometric constraint rather than a sensor model.

4.2 Bounded Covariance Growth in Local Maps

A crucial advantage of sequential map joining is that local map covariance matrices do not grow indefinitely. Unlike full SLAM where $\hat{P}$ expands with each new feature, local maps reach a saturation size determined by sensor range.

With finite sensor range r, only features within distance r from the robot are observable. In our 1D environment, as the robot moves forward, features behind exit the sensor's rear range while new features ahead enter the front range creating a an "observable window".

As Table 3 demonstrates, local map covariance matrices stabilize at a characteristic size that remains approximately constant across different local maps of equal trajectory length. Each local map processes the same number of motion steps, yet the final matrix dimensions vary only slightly around a mean value.

Table 3: Covariance matrix dimensions remain bounded in local maps

Maps	**Steps**	**Final Size $\hat{P}$**
1	1000	1003
2	1000	2003
3	1000	3003
4	1000	4003

5 Experimental Results

To validate the sequential map joining implementation, we compared the estimation performance against the full SLAM baseline. Figure 5 shows the robot localization error and uncertainty bounds for a single 4000 step trajectory using 1 map.



Figure 5: Robot estimation error with 2σ uncertainty bounds for full SLAM over 4000 steps.

The robot's position error (red line) remains consistently within the 2σ confidence bounds (blue lines)

throughout the trajectory, indicating that the filter provides a statistically consistent estimate of localization uncertainty.

Figure 6 presents the same analysis for the sequential map joining approach with 4 local maps of 1000 steps each.



Figure 6: Robot estimation error with 2σ uncertainty bounds for sequential map joining (4 maps $\times$ 1000 steps).

The sequential approach exhibits distinct behavior at map boundaries. Each local map begins with zero initial uncertainty (the robot starts at a known reference), which grows gradually within each 1000-step segment. At map joining events (visible as vertical discontinuities), the accumulated uncertainty from the previous map propagates forward, creating stepped increases in the global uncertainty envelope.

Despite these discontinuities, the estimation error remains bounded within the 2σ limits, confirming that:

1. The map joining procedure correctly propagates uncertainty across map boundaries
2. The duplicate feature fusion maintains filter consistency
3. The final localization accuracy is comparable to full 1 map experiment.

Team Contributions

This project was completed through an iterative collaborative process. Diego built the initial codebase with the core assignment functions, but early results showed numerical instabilities in both state estimates and performance metrics. Piyush debugged and stabilized the implementation, which was essential before moving forward with map joining.

With a stable foundation, both team members worked together the map joining algorithm, refining it to ensure correct uncertainty propagation. Piyush then implemented the duplicate feature fusion algorithm while Diego handled documentation, results analysis, and report writing. Both reviewed and refined the final content together.